

## Introduction to Theory of Computation

Automata theory, also known as the Theory of Computation, is a field within computer science and mathematics that focuses on studying abstract machines to understand the capabilities and limitations of computation by analyzing mathematical models of how machines can perform calculations.

### 1. Symbol

A symbol (often also called a character) is the smallest building block, which can be any alphabet, letter, or picture.

### 2. Alphabets ( $\Sigma$ )

A finite, non-empty set of symbols used to construct strings and languages. For example,  $\Sigma = \{a, b\}$ .

### 3. String

A string is a *finite* sequence of symbols from some alphabet. A string is generally denoted as  $w$  and the length of a string is denoted as  $|w|$ . Empty string is the string with zero occurrence of symbols, represented as  $\epsilon$ .

***Number of Strings (of length 2)  
that can be generated over the alphabet  $\{a, b\}$ :***

--

*a a*

*a b*

*b a*

*b b*

Length of String  $|w| = 2$

Number of Strings = 4

### Conclusion:

For alphabet  $\{a, b\}$  with length  $n$ , number of strings can be generated =  $2n$ .

Automata theory is used in modeling computational problems hence enhancing the understanding and design of systems such as compilers, interpreters among others.

### Closure Representation in TOC

1.  $L^+$ : It is a **Positive Closure** that represents a set of all strings except Null or  $\epsilon$ -strings.

2.  $L^*$ : It is "**Kleene Closure**", that represents the occurrence of certain alphabets for given language alphabets from zero to the infinite number of times. In which  $\epsilon$ -string is also included.

From the above two statements, it can be concluded that:  $L^* = \epsilon L^+$

### Example:

(a) Regular expression for language accepting all combination of  $g$ 's over  $\Sigma = \{g\}$ :

$R = g^*$

$R = \{\epsilon, g, gg, ggg, gggg, ggggg, gggggg, \dots\}$

(b) Regular Expression for language accepting all combination of  $g$ 's over  $\Sigma = \{g\}$ :

$R = g^+$

$R = \{g, gg, ggg, gggg, ggggg, gggggg, ggggggg, \dots\}$

Kleene Star is also called a "**Kleene Operator**" or "**Kleene Closure**". Engineers and IT professionals make use of Kleene Star to achieve all set of strings which is to be included from a given set of characters or symbols. It is one kind of Unary operator. In Kleene Star methodology all individual elements of a given string must be present but additional elements or combinations of these alphabets can be included to any extent.

### Example:

**Input String: "GFG".**

$\Sigma^* = \{$

$\epsilon, "GFG", "GGFG", "GGFG", "GFGGGGGGGG", "GGGGGGGGGGGGGGGGGGGGGGGGGGGGGG", \dots\}$

(Kleene Star is an infinite set but if we provide any grammar rules then it can work as a finite set.

Please note that we can include  $\epsilon$  string also in given Kleene star representation.)

## Language

- A language is a set of strings formed using the symbols of a given alphabet  $\Sigma$ .
- Formally, a language is a subset of  $\Sigma^*$ , where  $\Sigma^*$  is the set of all possible strings (including  $\epsilon$ ) over the alphabet  $\Sigma$ .

### Examples of Languages:

*Finite Language:*

$L_1 = \{ \text{set of string of 2} \}$

$L_1 = \{ xy, yx, xx, yy \}$

*Infinite Language:*

$L_1 = \{ \text{set of all strings starts with 'b'} \}$

$L_1 = \{ babb, baa, ba, bbb, baab, \dots \}$

### Types of Languages in TOC

Languages are classified based on the computational model or grammar generating them:

1. **Regular Languages:** Defined using regular expressions or finite automata.  
Example:  $L = a^n \mid n \geq 0$ .
2. **Context-Free Languages:** Defined using context-free grammars or pushdown automata. Example:  $L = a^n b^n \mid n \geq 0$ .
3. **Context-Sensitive Languages:** Defined using context-sensitive grammars or linear-bounded automata.
4. **Recursive and Recursively Enumerable Languages:** Defined using Turing machines.

## Introduction of Finite Automata

Finite automata are abstract machines used to recognize patterns in input sequences, forming the basis for understanding regular languages in computer science.

- Consist of states, transitions, and input symbols, processing each symbol step-by-step.
- If ends in an accepting state after processing the input, then the input is accepted; otherwise, rejected.
- Finite automata come in deterministic (DFA) and non-deterministic (NFA), both of which can recognize the same set of regular languages.
- Widely used in text processing, compilers, and network protocols.

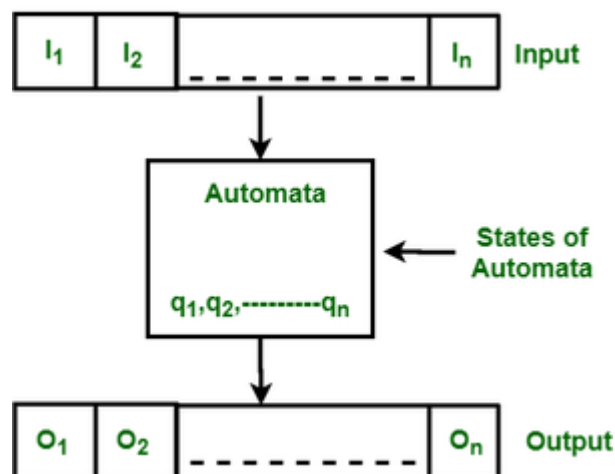


Figure: Features of Finite Automata

### Features of Finite Automata

- **Input:** Set of symbols or characters provided to the machine.
- **Output:** Accept or reject based on the input pattern.
- **States of Automata:** The conditions or configurations of the machine.
- **State Relation:** The transitions between states.
- **Output Relation:** Based on the final state, the output decision is made.

## Formal Definition of Finite Automata

A finite automaton can be defined as a tuple:

$\{ Q, \Sigma, q, F, \delta \}$ , where:

- $Q$ : Finite set of states
- $\Sigma$ : Set of input symbols
- $q$ : Initial state
- $F$ : Set of final states
- $\delta$ : Transition function

## Types of Finite Automata

There are two types of finite automata:

1. Deterministic Finite Automata (DFA)
2. Non-Deterministic Finite Automata (NFA)

### 1. Deterministic Finite Automata (DFA)

A DFA is represented as  $\{Q, \Sigma, q, F, \delta\}$ . In DFA, for each input symbol, the machine transitions to one and only one state. DFA does not allow any null transitions, meaning every state must have a transition defined for every input symbol.

*DFA consists of 5 tuples  $\{Q, \Sigma, q, F, \delta\}$ .*

*$Q$  : set of all states.*

*$\Sigma$  : set of input symbols. ( Symbols which machine takes as input )*

*$q$  : Initial state. ( Starting state of a machine )*

*$F$  : set of final state.*

*$\delta$  : Transition Function, defined as  $\delta : Q \times \Sigma \rightarrow Q$ .*

**Example: Construct a DFA that accepts all strings ending with 'a'.**

**Given:**

$\Sigma = \{a, b\}$ ,

$Q = \{q_0, q_1\}$ ,

$F = \{q_1\}$

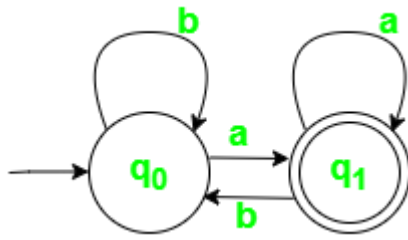


Fig 1. State Transition Diagram for DFA with  $\Sigma = \{a,$

$b\}$

| State \ Symbol | a              | b              |
|----------------|----------------|----------------|
| q <sub>0</sub> | q <sub>1</sub> | q <sub>0</sub> |
| q <sub>1</sub> | q <sub>1</sub> | q <sub>0</sub> |

In this example, if the string ends in 'a', the machine reaches state q<sub>1</sub>, which is an accepting state.

## 2. Non-Deterministic Finite Automata (NFA)

NFA is similar to DFA but includes the following features:

- It can transition to multiple states for the same input.
- It allows null ( $\epsilon$ ) moves, where the machine can change states without consuming any input.

Example: Construct an NFA that accepts strings ending in 'a'.

Given:

$\Sigma = \{a, b\},$

$Q = \{q_0, q_1\},$

$F = \{q_1\}$

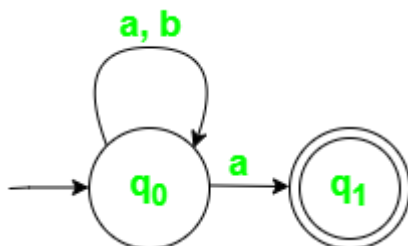


Fig 2. State Transition Diagram for NFA with  $\Sigma = \{a, b\}$

**State Transition Table for above Automaton,**

| State \ Symbol | a                                 | b              |
|----------------|-----------------------------------|----------------|
| q <sub>0</sub> | {q <sub>0</sub> ,q <sub>1</sub> } | q <sub>0</sub> |
| q <sub>1</sub> | ϕ                                 | ϕ              |

**In an NFA, if any transition leads to an accepting state, the string is accepted.**

### **NFA vs DFA**

| <b>DFA</b>                                                        | <b>NFA</b>                                                              |
|-------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Deterministic: exactly one transition per input symbol</b>     | <b>Non-deterministic: multiple transitions per input symbol allowed</b> |
| <b>No <math>\epsilon</math> (null) moves</b>                      | <b>Allows <math>\epsilon</math> (null) moves</b>                        |
| <b>Simpler but sometimes larger in design</b>                     | <b>More flexible and easier to design</b>                               |
| <b>Next state is uniquely determined</b>                          | <b>Next state may be multiple possibilities</b>                         |
| <b>Recognizes regular languages; NFAs can be converted to DFA</b> | <b>Recognizes regular languages; equivalent in power to DFA</b>         |

## Applications of Finite Automata

The following table summarizes the key applications of finite automata –

| Application Area           | Description                                                                            |
|----------------------------|----------------------------------------------------------------------------------------|
| Language Processing        | Used in email filters and smart assistants like Alexa, Siri, and Google Assistant.     |
| Compiler Construction      | Translates high-level programming languages into machine code.                         |
| Computer Networks          | Underlies the design of network protocols such as HTTP and HTTPS.                      |
| Video Games                | Manages game mechanics and AI, as seen in games like Warcraft 3.                       |
| Digital Circuit Design     | Designs digital circuits in devices such as fans, refrigerators, and washing machines. |
| Biomedical Problem Solving | Utilized in medical imaging technologies like X-ray diffraction and tomography.        |

The above table is for quick reference. Let us understand them in detail one by one.

### Finite Automata in Language Processing

One of the most important applications of automata is "language processing". It involves the interaction between human language and computational systems. We can further discuss the key applications in this domain –

- **Email Filters** – Finite automata help to differentiate between legitimate emails and the spams, ensuring users only see relevant messages.
- **Smart Assistants** – Devices like Amazon's Alexa, Microsoft's Cortana, Apple's Siri, and Google Assistant and other voice assistance tools use finite automata algorithms for interpreting and responding to user inputs.

### Finite Automata in Compiler Construction

After language processing, it is necessary to talk about compilers. In compiler construction, finite automata are crucial for translating high-level programming languages into machine code. FA enables computers to execute complex programs by converting them into a language they understand.

#### Finite Automata in Computer Networks

There are some great applications in computer networks also. These are essential in developing protocols for computer networks. For example –

**Network Protocols** – Protocols such as HTTP and HTTPS rely on finite automata for managing data transmission and communication across networks, ensuring data is transferred accurately and securely.

#### Finite Automata in Video Games

Video games often utilize finite automata for managing complex algorithms and game mechanics. For instance: The algorithm design for games like Warcraft 3, they use finite automata to create advanced AI behaviours, enhancing the player's gaming experience.

#### Finite Automata in Digital Circuit Design

Finite automata are used in designing digital circuits as well, which are fundamental components of electronic devices. In circuit designing, everyday devices like fans, refrigerators, and washing machines incorporate digital circuits designed using finite automata, ensuring their efficient and reliable operation.

#### Finite Automata in Biomedical Problem Solving

In biomedical fields, the finite automata also play a great role, particularly in medical imaging, where technologies such as X-ray diffraction and X-ray tomography utilize finite automata algorithms to produce accurate diagnostic images.

### Example 1

Let a deterministic finite automaton be →

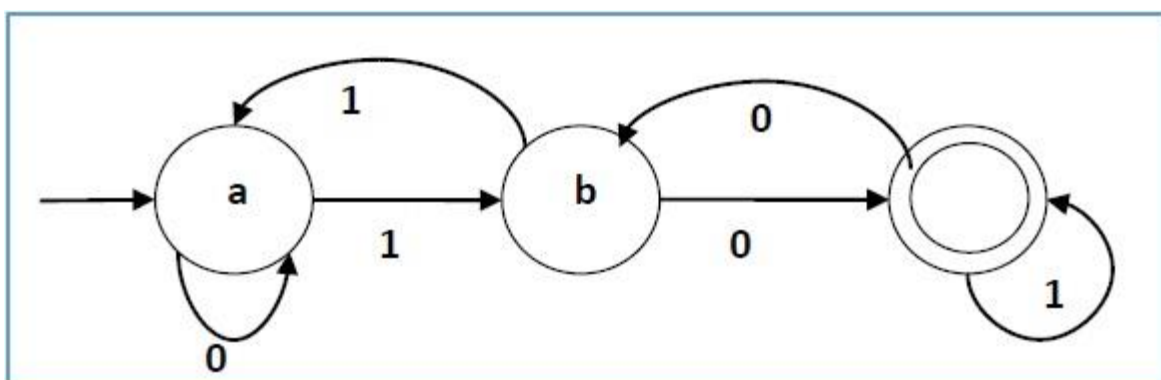
- $Q = \{a, b, c\}$ ,
- $\Sigma = \{0, 1\}$ ,
- $q_0 = \{a\}$ ,
- $F = \{c\}$ , and

### Transition Table

Transition function  $\delta$  as shown by the following table –

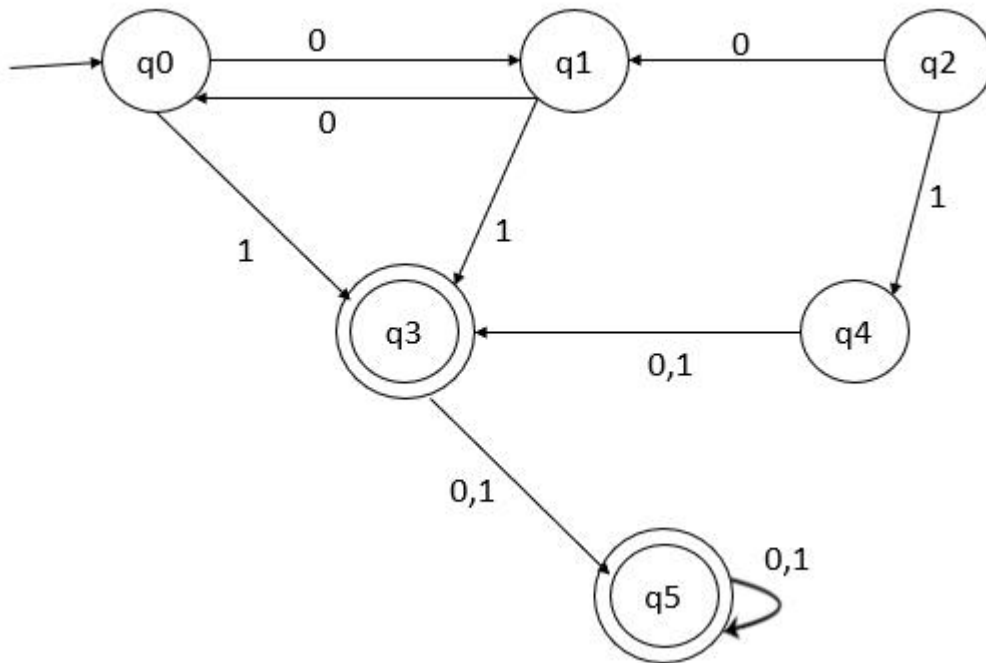
| Present State | Next State for Input 0 | Next State for Input 1 |
|---------------|------------------------|------------------------|
| <b>a</b>      | a                      | b                      |
| <b>b</b>      | c                      | a                      |
| <b>c</b>      | b                      | c                      |

Its graphical representation would be as follows –



### Example 3

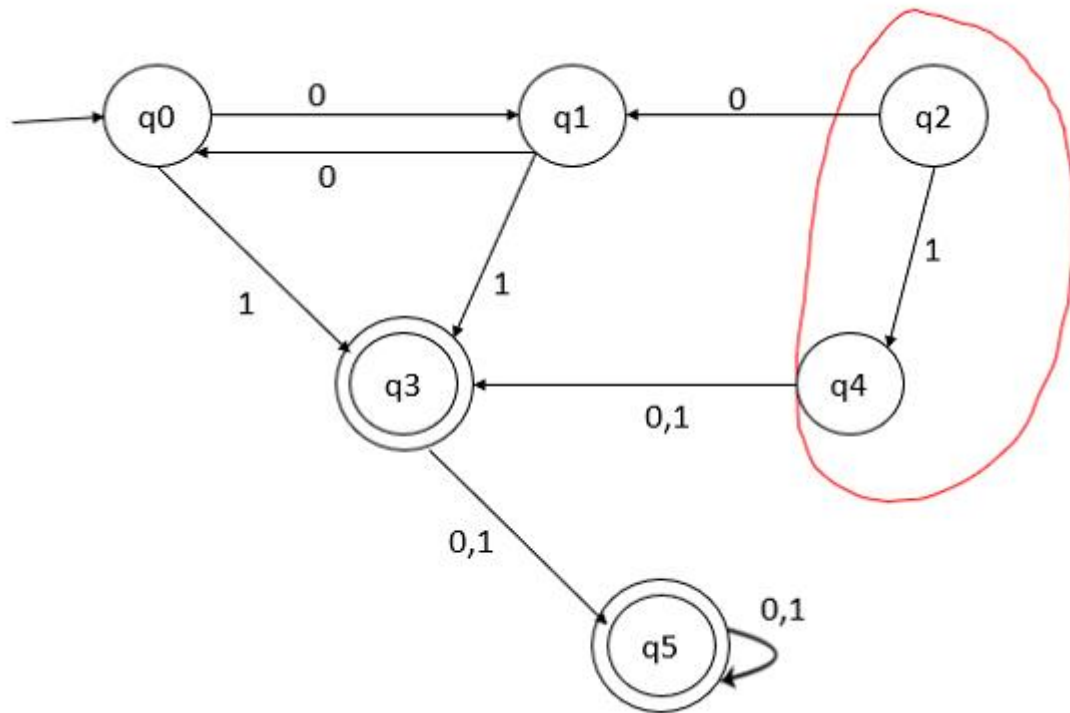
Given a Deterministic Finite Automata (DFA), try to reduce the DFA by removing unreachable states and removing similar rows.



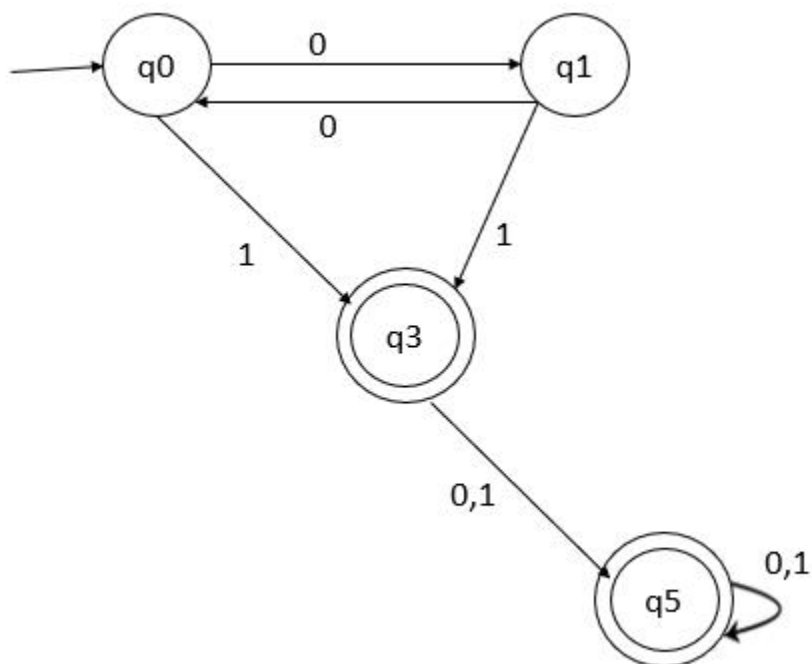
### Solution

**Step 1** – Remove the unreachable states from q0

From the initial states, we are not able to reach q2 and q4. So, remove these two states as shown below –



After removing unreachable states, the partial minimized DFA is as follows –



**Step 2** – The **transition table** is given below –

| States | 0 | 1 |
|--------|---|---|
|        |   |   |

|      |    |    |
|------|----|----|
| → q0 | q1 | q3 |
| q1   | q0 | q3 |
| *q3  | q5 | q5 |
| *q5  | q5 | q5 |

**Step 3** – Divide tables into 2 tables as shown below –

**Table 1** starts from the non-final states.

| States | 0  | 1  |
|--------|----|----|
| → q0   | q1 | q3 |
| q1     | q0 | q3 |

**Table 2** starts from the final states.

| States | 0  | 1  |
|--------|----|----|
| *q3    | q5 | q5 |
| *q5    | q5 | q5 |

**Step 4** – Remove similar rows.

Table 1 has no similar rows

Table 2 has similar rows. So, skip q5 and replace q5 by q3

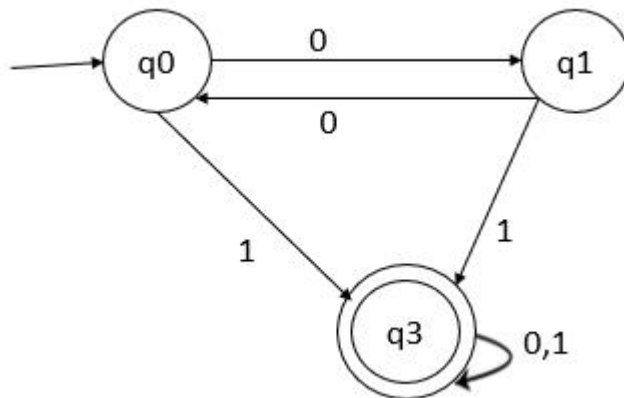
| States | 0  | 1  |
|--------|----|----|
| q3     | q5 | q3 |

**Step 5** – Combine two tables as shown below –

| States | 0 | 1 |
|--------|---|---|
|--------|---|---|

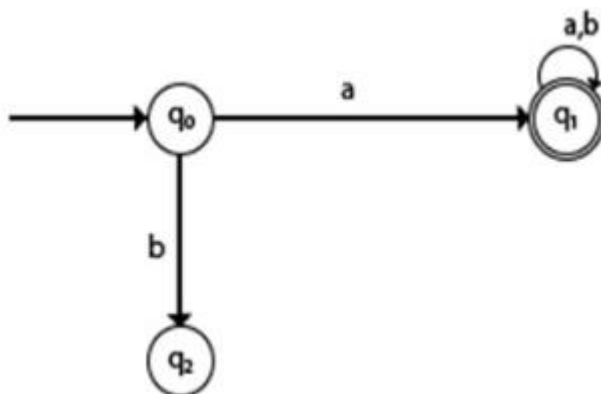
|                   |       |       |
|-------------------|-------|-------|
| $\rightarrow q_0$ | $q_1$ | $q_3$ |
| $q_1$             | $q_0$ | $q_3$ |
| $*q_3$            | $q_3$ | $q_3$ |

Thus, the **minimized DFA** will be as follows –



Trap State in DFA

If a transition goes to a state from which it can never escape. Such a state is called a trap state. It is called the dead state.



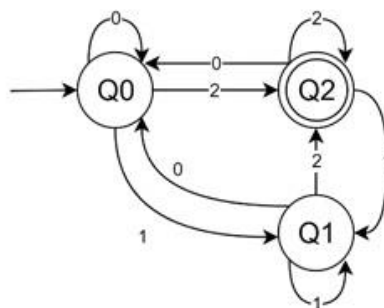
In the above example,  $q_2$  is a trap or dead state because it can't reach the final state.

## Example of Involving an Elevator System with Three Floors

To better understand finite automata, let's consider a simple example involving a lift (elevator) system with three floors: Ground floor, First floor, and Second floor.

The lift can move between these floors based on user input. To design the automata, we need to first define the states, then the alphabets, transition, initial, state and the final state. Let's discuss each of them one by one –

- **States (Q):**
  - **Q0:** Ground floor
  - **Q1:** First floor
  - **Q2:** Second floor
- **Input Alphabets ( $\Sigma$ ):**
  - **0:** Go to the ground floor
  - **1:** Go to the first floor
  - **2:** Go to the second floor
- **Transition System** – The transition system of the lift can be represented as a directed graph.



In this directed graph, **circles** denote states (Q0, 1, Q2). **Directed edges** represent transitions based on input symbols.

The transition system of the lift can be visualized as the above mentioned directed graph with three states: Q0 (ground floor), Q1 (first floor), and Q2 (second floor).

Each state is represented by a circle. The initial state, Q0, is indicated by an inward arrow. Transitions between states are depicted by directed edges based on input symbols.

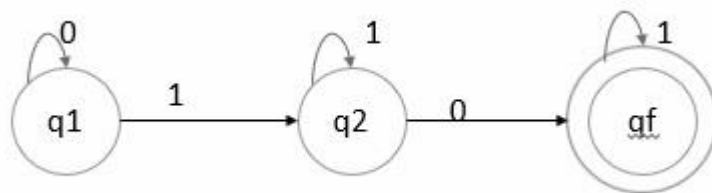
- From 0, an input of 0 leads to Q0 (self-loop), an input of 1 leads to Q1, and an input of 2 leads to Q2.

- From 1, an input of 0 transitions to Q0, an input of 1 remains in Q1 (self-loop), and an input of 2 transitions to Q2.
- From 2, an input of 0 transitions to Q0, an input of 1 transitions to Q1, and an input of 2 remains in Q2 (self-loop).
- The final state, 2, is represented by a double circle.

### Transition Diagram

It is a directed graph associated with the vertices of the graph corresponding to the state of finite automata.

An example of transition diagram is given below –



Here,

- {0,1}: Inputs
- q1: Initial state
- q2: Intermediate state
- qf: Final state

### Transition Table

We can also write the same through a **transition table** –

| Present State | Input | Next State |
|---------------|-------|------------|
| Q0            | 0     | Q0         |
| Q0            | 1     | Q1         |
| Q0            | 2     | Q2         |
| Q1            | 0     | Q0         |

|    |   |    |
|----|---|----|
| Q1 | 1 | Q1 |
| Q1 | 2 | Q2 |
| Q2 | 0 | Q0 |
| Q2 | 1 | Q1 |
| Q2 | 2 | Q2 |

### Example of NFA

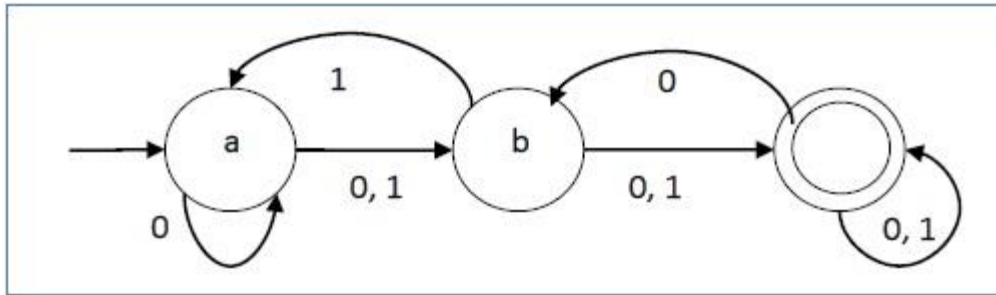
Let a non-deterministic finite automaton be →

- $Q = \{a, b, c\}$
- $\Sigma = \{0, 1\}$
- $q_0 = \{a\}$
- $F = \{c\}$

The transition function  $\delta$  as shown below –

| Present State | Next State for Input 0 | Next State for Input 1 |
|---------------|------------------------|------------------------|
| a             | a, b                   | b                      |
| b             | c                      | a, c                   |
| c             | b, c                   | c                      |

Its graphical representation would be as follows –



## NDFA to DFA Conversion

### Conversion from NFA to DFA

An NFA can have zero, one or more than one move from a given state on a given input symbol. An NFA can also have NULL moves (moves without input symbol). On the other hand, DFA has one and only one move from a given state on a given input symbol.

Steps for converting NFA to DFA:

Step 1: Convert the given NFA to its equivalent transition table

To convert the NFA to its equivalent transition table, we need to list all the states, input symbols, and the transition rules. The transition rules are represented in the form of a matrix, where the rows represent the current state, the columns represent the input symbol, and the cells represent the next state.

Step 2: Create the DFA's start state

The DFA's start state is the set of all possible starting states in the NFA. This set is called the "epsilon closure" of the NFA's start state. The epsilon closure is the set of all states that can be reached from the start state by following epsilon (?) transitions.

Step 3: Create the DFA's transition table

The DFA's transition table is similar to the NFA's transition table, but instead of individual states, the rows and columns represent sets of states. For each input symbol, the corresponding cell in the transition table contains the epsilon closure of the set of states obtained by following the transition rules in the NFA's transition table.

Step 4: Create the DFA's final states

The DFA's final states are the sets of states that contain at least one final state from the NFA.

Step 5: Simplify the DFA

The DFA obtained in the previous steps may contain unnecessary states and transitions. To simplify the DFA, we can use the following techniques:

- Remove unreachable states: States that cannot be reached from the start state can be removed from the DFA.
- Remove dead states: States that cannot lead to a final state can be removed from the DFA.
- Merge equivalent states: States that have the same transition rules for all input symbols can be merged into a single state.

Step 6: Repeat steps 3-5 until no further simplification is possible

After simplifying the DFA, we repeat steps 3-5 until no further simplification is possible.

The final DFA obtained is the minimized DFA equivalent to the given NFA.

Example: Consider the following NFA shown in Figure 1.

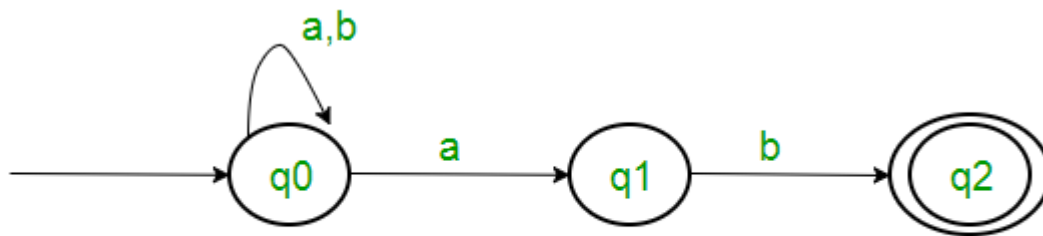


Figure 1

Following are the various parameters for NFA.  $Q = \{ q_0, q_1, q_2 \}$   $\Sigma = ( a, b )$   $F = \{ q_2 \}$   $\delta$   
(Transition Function of NFA)

| State | a     | b  |
|-------|-------|----|
| q0    | q0,q1 | q0 |
| q1    |       | q2 |
| q2    |       |    |

Step 1:  $Q' = \emptyset$  Step 2:  $Q' = \{ q_0 \}$  Step 3: For each state in  $Q'$ , find the states for each input symbol. Currently, state in  $Q'$  is  $q_0$ , find moves from  $q_0$  on input symbol  $a$  and  $b$  using transition function of NFA and update the transition table of DFA.  $\delta'$  = Transition Function of DFA

| State | a       | b  |
|-------|---------|----|
| q0    | {q0,q1} | q0 |

Now { q0, q1 } will be considered as a single state. As its entry is not in Q', add it to Q'.

So  $Q' = \{ q0, \{ q0, q1 \} \}$

Now, moves from state { q0, q1 } on different input symbols are not present in transition table of DFA, we will calculate it like:

- $\delta'(\{ q0, q1 \}, a) = \delta(q0, a) \cup \delta(q1, a) = \{ q0, q1 \}$
- $\delta'(\{ q0, q1 \}, b) = \delta(q0, b) \cup \delta(q1, b) = \{ q0, q2 \}$

Now we will update the transition table of DFA.  $\delta'$  (Transition Function of DFA)

| State   | a       | B       |
|---------|---------|---------|
| q0      | {q0,q1} | q0      |
| {q0,q1} | {q0,q1} | {q0,q2} |

Now { q0, q2 } will be considered as a single state. As its entry is not in Q', add it to Q'. So  $Q' = \{ q0, \{ q0, q1 \}, \{ q0, q2 \} \}$  Now, moves from state {q0, q2} on different input symbols are not present in transition table of DFA, we will calculate it like:

- $\delta'(\{ q0, q2 \}, a) = \delta(q0, a) \cup \delta(q2, a) = \{ q0, q1 \}$
- $\delta'(\{ q0, q2 \}, b) = \delta(q0, b) \cup \delta(q2, b) = \{ q0 \}$

Now we will update the transition table of DFA.  $\delta'$  (Transition Function of DFA)

| State   | a       | B       |
|---------|---------|---------|
| q0      | {q0,q1} | q0      |
| {q0,q1} | {q0,q1} | {q0,q2} |
| {q0,q2} | {q0,q1} | q0      |

As there is no new state generated, we are done with the conversion. Final state of DFA will be state which has q2 as its component i.e., { q0, q2 } Following are the various parameters for DFA.

- $Q' = \{ q0, \{ q0, q1 \}, \{ q0, q2 \} \}$
- $\Sigma = \{ a, b \}$
- $F = \{ \{ q0, q2 \} \}$
- $\delta'$  as shown above

The final DFA for above NFA has been shown in Figure 2.

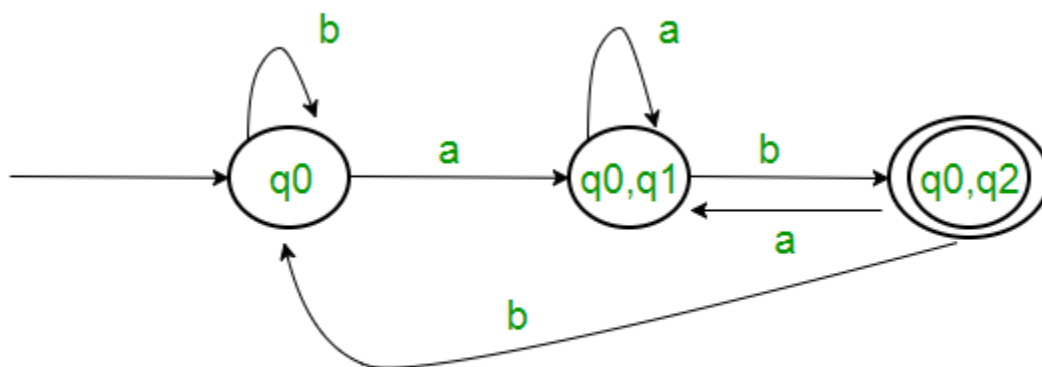
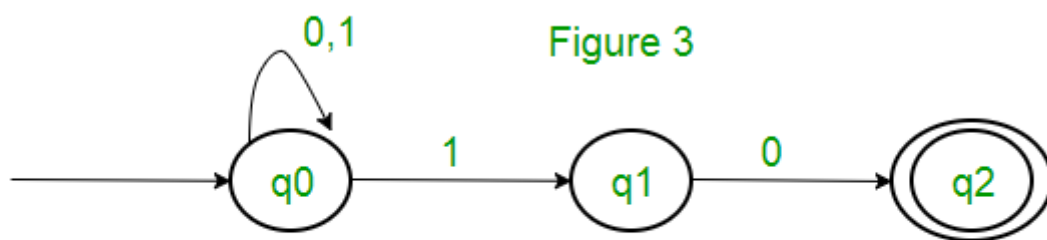


Figure 2

Note : Sometimes, it is not easy to convert regular expression to DFA. First you can convert regular expression to NFA and then NFA to DFA.

Question : The number of states in the minimal deterministic finite automaton corresponding to the regular expression  $(0 + 1)^* (10)$  is \_\_\_\_\_.

Solution : First, we will make an NFA for the above expression. To make an NFA for  $(0 + 1)^*$ , NFA will be in same state  $q_0$  on input symbol 0 or 1. Then for concatenation, we will add two moves ( $q_0$  to  $q_1$  for 1 and  $q_1$  to  $q_2$  for 0) as shown in Figure 3.



Using above algorithm, we can convert NFA to DFA as shown in Figure 4

| State | 0     | 1     |
|-------|-------|-------|
| q0    | q0    | q0,q1 |
| q0,q1 | q0,q2 | q0,q1 |
| q0,q2 | q0    | q0,q1 |

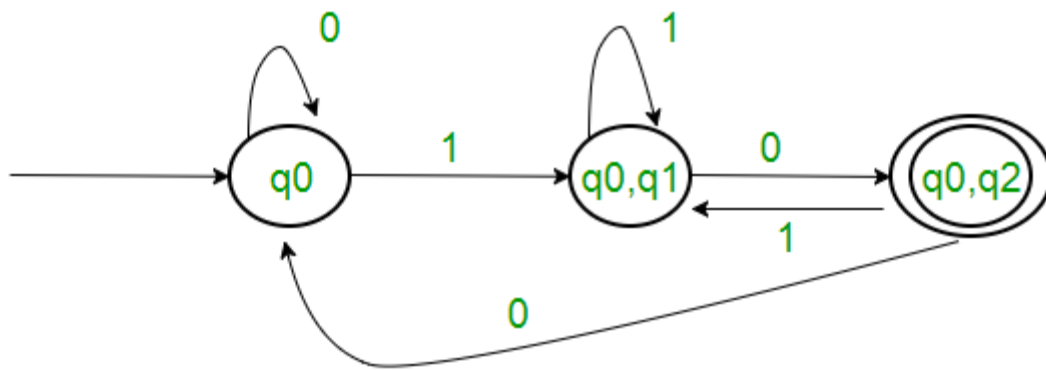


Figure 4

### Problem Statement

Let  $X = (Q_x, \Sigma, \delta_x, q_0, F_x)$  be an NFA which accepts the language  $L(X)$ . We have to design an equivalent DFA  $Y = (Q_y, \Sigma, \delta_y, q_0, F_y)$  such that  $L(Y) = L(X)$ . The following procedure converts the NFA to its equivalent DFA –

### Algorithm

**Input** – An NFA

**Output** – An equivalent DFA

**Step 1** – Create state table from the given NFA.

**Step 2** – Create a blank state table under possible input alphabets for the equivalent DFA.

**Step 3** – Mark the start state of the DFA by  $q_0$  (Same as the NFA).

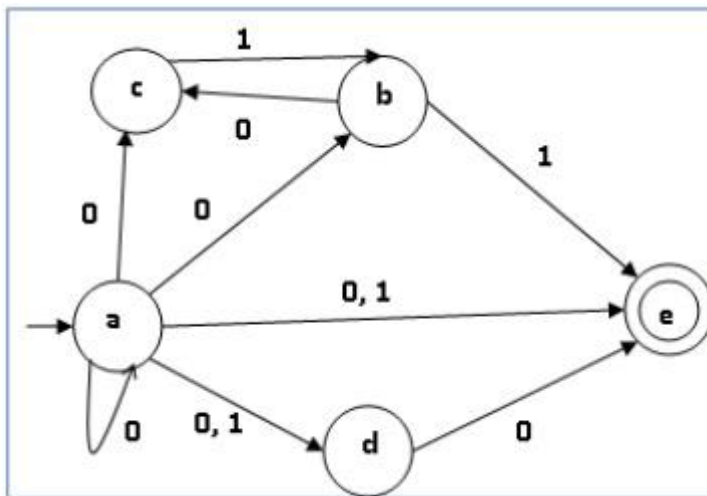
**Step 4** – Find out the combination of States  $\{Q_0, Q_1, \dots, Q_n\}$  for each possible input alphabet.

**Step 5** – Each time we generate a new DFA state under the input alphabet columns, we have to apply step 4 again, otherwise go to step 6.

**Step 6** – The states which contain any of the final states of the NFA are the final states of the equivalent DFA.

### Example

Let us consider the N DFA shown in the figure below.



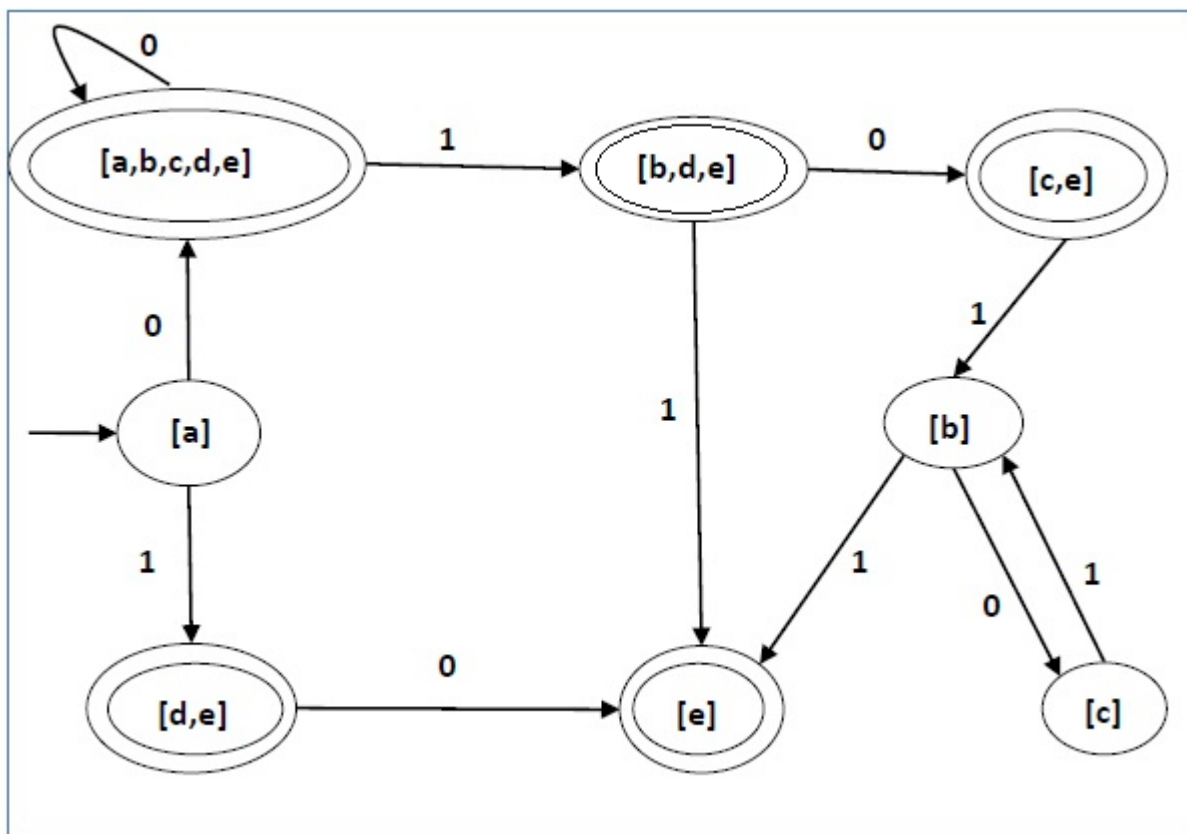
| q | (q,0)       | (q,1)       |
|---|-------------|-------------|
| a | {a,b,c,d,e} | {d,e}       |
| b | {c}         | {e}         |
| c | $\emptyset$ | {b}         |
| d | {e}         | $\emptyset$ |
| e | $\emptyset$ | $\emptyset$ |

Using the above algorithm, we find its equivalent DFA. The state table of the DFA is shown in below.

| q           | $\delta(q,0)$ | $\delta(q,1)$ |
|-------------|---------------|---------------|
| [a]         | [a,b,c,d,e]   | [d,e]         |
| [a,b,c,d,e] | [a,b,c,d,e]   | [b,d,e]       |
| [d,e]       | [e]           | $\emptyset$   |
| [b,d,e]     | [c,e]         | [e]           |

|        |             |             |
|--------|-------------|-------------|
| [e]    | $\emptyset$ | $\emptyset$ |
| [c, e] | $\emptyset$ | [b]         |
| [b]    | [c]         | [e]         |
| [c]    | $\emptyset$ | [b]         |

The state diagram of the DFA is as follows –



### Equivalence of NFA and DFA

**Finite Automata** are basic model in automata theory that are simple because of their finiteness. The finite automata may have in two forms the deterministic and non-deterministic.

Finite Automata are used to recognize patterns and define languages. A critical concept within this domain is the equivalence of two finite automata. It is proven that two types of finite automata perform the same function by accepting the same language.

The Concept of Equivalence

Equivalence between two finite automata means that both automata accept the same language. In other words, for any given input string, both automata either accept or reject it consistently. But there are some conditions when we are going to check equivalence of two automata.

- **State Pair Comparison** – For any pair of states, say  $(q_i, q_j)$  from two different automata, the transitions for an input  $A$  belonging to the input alphabet  $\Sigma$  must lead to states  $q_A$  and  $q_B$ , respectively. If one state in the pair is a final state and the other is not, the automata are not equivalent.
- **Initial and Final State Consistency** – If the initial state of one automaton is also a final state, the initial state of the second automaton must also be a final state for them to be equivalent.

#### Steps to Determine Equivalence

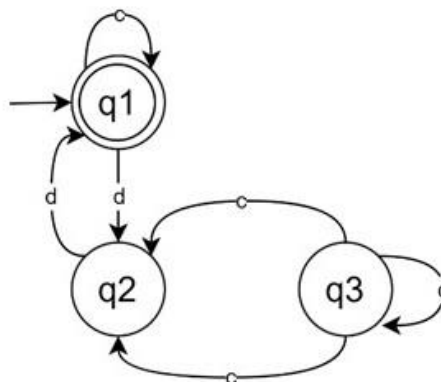
Let us see the steps involved to check whether two finite automata are equivalent or not.

- **Identify State Pairs** – Start by pairing the initial states of both automata.
- **Transition Analysis** – For each pair, analyze and check the transitions for all inputs in the alphabet  $\Sigma$ .
- **State Comparison** – Compare the resultant states from the transitions. Then ensure both resultant states are either final or non-final (intermediate).
- **Repeat for All Pairs** – Continue the process for all possible pairs of states.

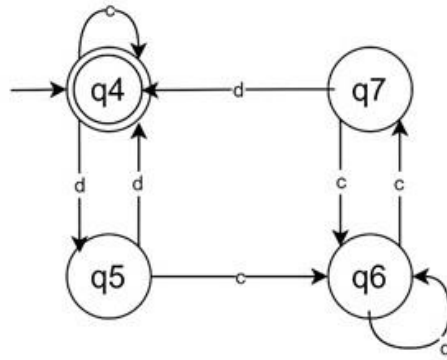
#### Checking Equivalence of Finite Automata

Consider two finite automata, **A** and **B**, with states and transitions as illustrated below. The inputs for both automata are  $c$  and  $d$ , respectively.

Consider the **first finite automata (A)** –



Consider the **second finite automata (B)** –



Now let us analyse the equivalence in steps –

- The initial pair is  $(q_1, q_4)$ 
  - For input  $c$ ,  $q_1 \rightarrow q_2$  and  $q_4 \rightarrow q_4$ , both are final states
  - For input  $d$ ,  $q_1 \rightarrow q_2$  and  $q_4 \rightarrow q_5$ , both are intermediate states
- Next pair is  $(q_2, q_5)$ 
  - For input  $c$ ,  $q_2 \rightarrow q_3$  and  $q_5 \rightarrow q_6$ , both are intermediate states
  - For input  $d$ ,  $q_2 \rightarrow q_1$  and  $q_5 \rightarrow q_4$ , both are final states
- Next pair is  $(q_3, q_6)$ 
  - For input  $c$ ,  $q_3 \rightarrow q_2$  and  $q_6 \rightarrow q_7$ , both are intermediate states
  - For input  $d$ ,  $q_3 \rightarrow q_3$  and  $q_6 \rightarrow q_6$ , both are intermediate states
- Next pair is  $(q_2, q_7)$ 
  - For input  $c$ ,  $q_2 \rightarrow q_3$  and  $q_7 \rightarrow q_6$ , both are intermediate states
  - For input  $d$ ,  $q_2 \rightarrow q_1$  and  $q_7 \rightarrow q_4$ , both are final states

From the above equivalence, we have compared between two machines **A** and **B**. Upon comparing all relevant pairs and their transitions, we find that for each pair, both resultant states are consistently either final or intermediate.

Therefore, based on the given conditions and steps, automata **A** and **B** are equivalent, as they meet all criteria for equivalence.

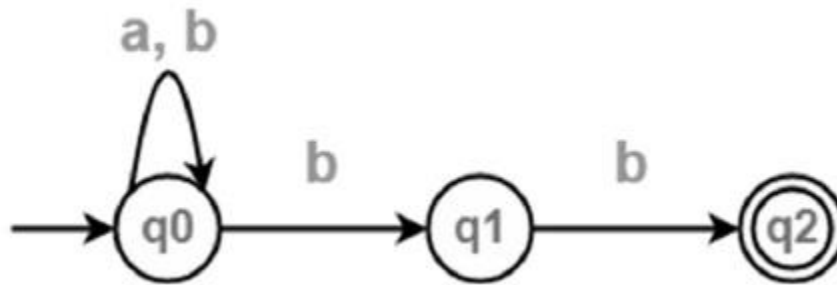
Examples: How to Convert NFA to DFA?

Example 1

Convert the following Non-Deterministic Finite Automata (NFA) to Deterministic Finite Automata (DFA).

Solution

The transition diagram is as follows –



The transition table of NFA is as follows –

| State | a  | b     |
|-------|----|-------|
| →q0   | q0 | q0,q1 |
| q1    | -  | *q2   |
| *q2   | -  | -     |

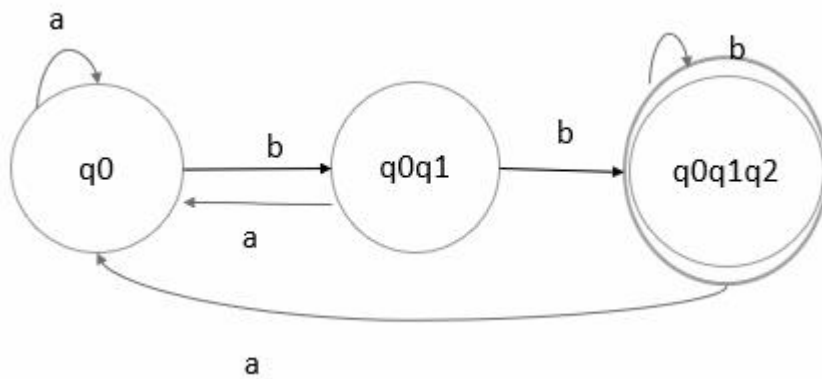
The DFA table cannot have multiple states. So, make q0q1 as a single state.

Let's convert the given NFA to DFA by considering two states as a single state.

The transition table of DFA is as follows –

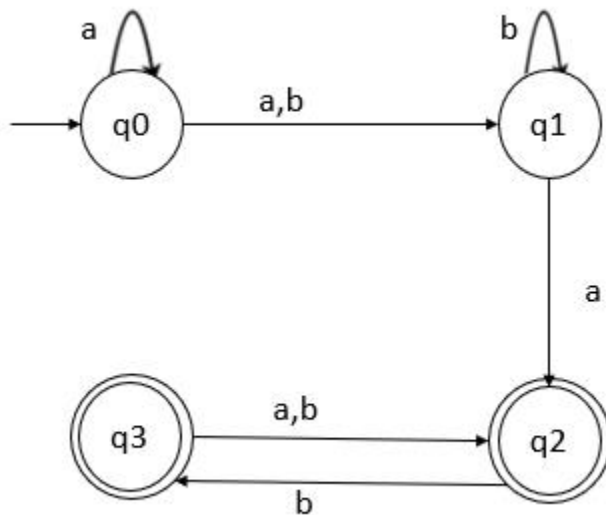
| State     | a    | b        |
|-----------|------|----------|
| → q0      | q0   | [q0,q1]  |
| [q0q1]    | [q0] | [q0q1q2] |
| *[q0q1q2] | [q0] | [q0q1q2] |

In the above transition table, q2 is the final state. Wherever, q2 is present that becomes the final state.



### Example 2

Consider a Non-deterministic finite automata (NFA) and convert that NFA into equivalent Deterministic Finite Automata (DFA).



### Solution

Let's construct NFA transition table for the given diagram –

| States\inputs | a       | b  |
|---------------|---------|----|
| →q0           | {q0,q1} | q0 |
| q1            | q2      | q1 |
| q2            | q3      | q3 |

|    |   |    |
|----|---|----|
| q3 | - | q2 |
|----|---|----|

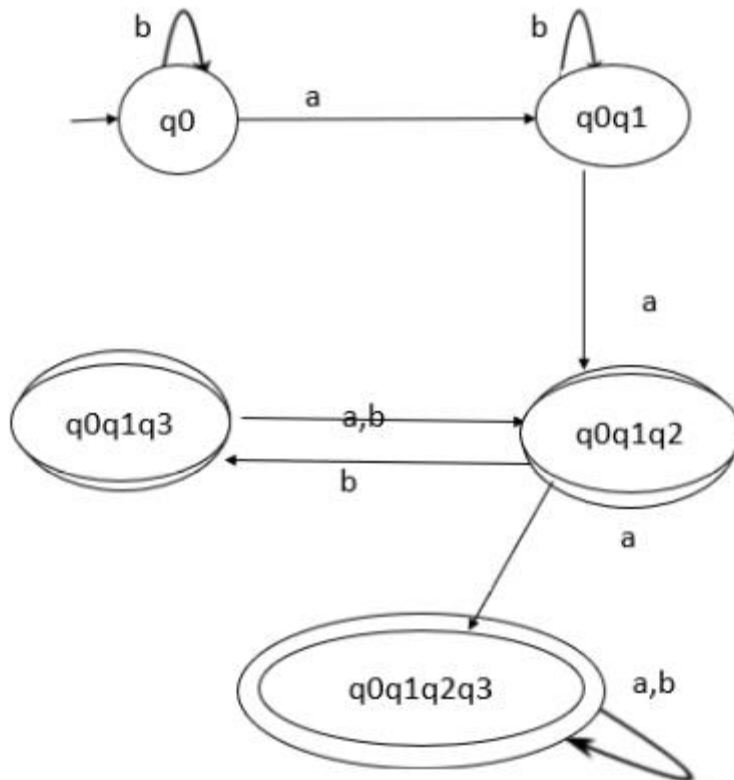
DFA cannot have multiple states. So, consider {q0,q1} as a single state while constructing DFA.

Let's convert the above table into equivalent DFA

| States\inputs | a          | b          |
|---------------|------------|------------|
| → q1          | [q0,q1]    | q0         |
| [q0,q1]       | [q0q1q2]   | [q0q1]     |
| *[q0q1q2]     | [q0q1q2q3] | [q0q1q3]   |
| *[q0q1q2q3]   | [q0q1q2q3] | [q0q1q2q3] |
| *[q0q1q3]     | [q0q1q2]   | [q0q1q2]   |

In DFA the final states are q2 and q3, wherever q2 and q3 are present that state becomes a final state.

Now the **transition diagram for DFA** is as follows –



- After conversion the number of states in the final DFA may or may not be the same as in NFA.
- The maximum number of states present in DFA may be  $2^{\text{pow}}(\text{number of states in NFA})$
- The relationship between number of states in NFA and DFA is:  $1 \leq n \leq 2^m$

Where **n** = number of states in DFA

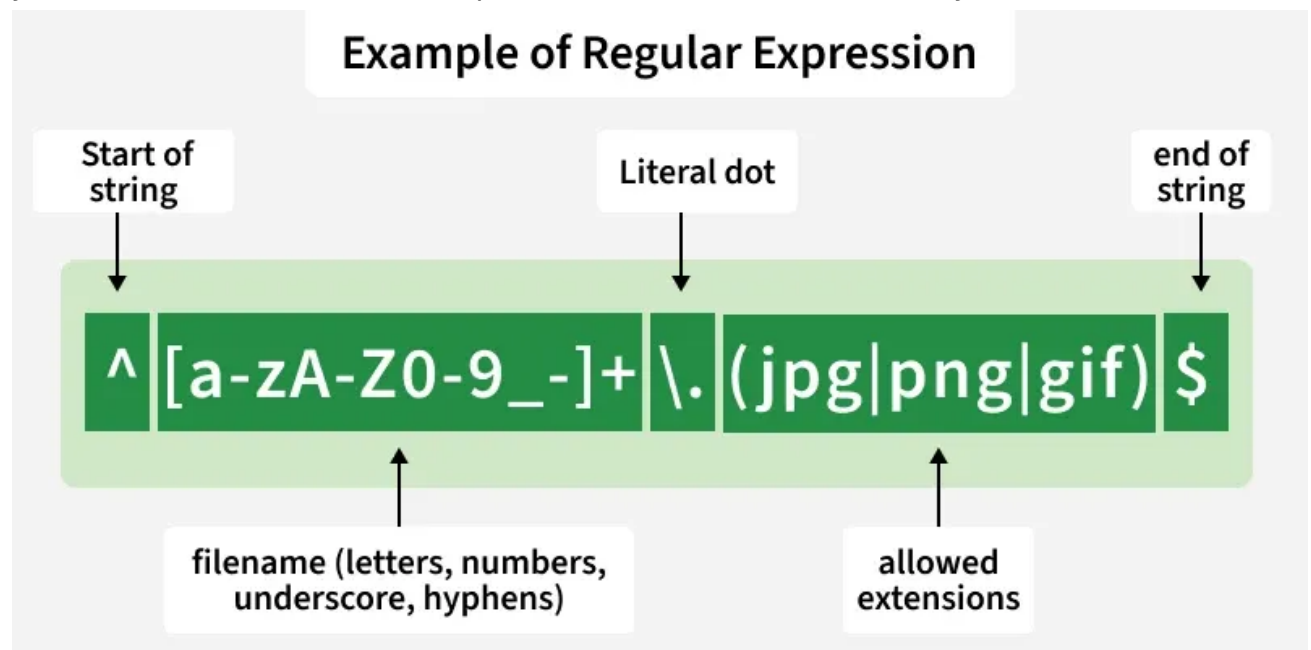
**m** = number of states in NFA

- The final DFA all states that contain the final states of NFA are treated as final states.

## How to write Regular Expressions

A regular expression (regex) is a sequence of characters that defines a search pattern. It is mainly used for pattern matching in strings, such as finding, replacing, or validating text. Regex is supported in almost every programming language, including Python, Java, C++ and JavaScript.

Below image shows an example of a regular expression and explains its parts, helping you understand how filenames or patterns can be matched effectively.



### Example of Regular Expression

This regex checks if a filename is valid, allowing letters, numbers, underscore, hyphens and ends with .jpg, .png or .gif. Example matches: file123.jpg, my-photo.png, logo\_1.gif.

**Examples:** Match a Filename Ending with .jpg, .png, or .gif

```
import re

pattern = r'^[a-zA-Z0-9_-]+\.(jpg|png|gif)$'

filename = "file123.jpg"

if re.match(pattern, filename):
    print("Valid image filename")
else:
```

```
print("Invalid filename")
```

## Importance of Regular Expression

1. **Efficient Pattern Matching:** Quickly search for specific patterns in text or data without manual checking.
2. **Data Validation:** Validate inputs like email addresses, phone numbers, URLs, and passwords.
3. **Text Manipulation:** Perform search-and-replace operations across files or datasets effectively.
4. **Automation in Analytics and Tools:** Used in tools like Google Analytics for URL matching and filtering.
5. **Cross-Platform Support:** Works across programming languages and editors such as Python, Java, Sublime Text, Notepad++, and Microsoft Word.

## Common Elements Used in Regular Expressions

Regular expressions are built using special symbols and characters. Below are the most commonly used regex elements explained with simple examples.

1. **Repeaters ( \*, +, and { } ):** Repeaters specify how many times the preceding character or group should appear.

2. **Asterisk symbol (\*):** Matches the preceding character 0 or more times.

**Example:** The regular expression *ab\*c* will give *ac, abc, abbc, abbbc....and so on*

3. **The Plus symbol (+):** Matches the preceding character 1 or more times.

**Example:** The regular expression *ab+c* will give *abc, abbc, abbbc, ... and so on.*

4. **The curly braces { ... }:** Defines an exact or range of repetitions.

**Example:** *{{2}: exactly 2 times*

*{min,}: at least min times*

*{min,max}: between min and max times*

5. **Wildcard (.):** Matches any single character except a newline.

**Example:** Regular expression *.\** will tell the computer that any character can be used any number of times.

6. **Optional character (?):** Matches 0 or 1 occurrence of the preceding character.

**Example:** *docx?* matches *doc* and *docx*

**7. The caret ( ^ ) symbol:** Ensures the match starts **at the beginning** of the string.

**Example :** `^\d{3}` matches 901 in 901-333

**8. The dollar ( \$ ) symbol:** Ensures the match ends at the end of the string.

**Example:** `\d{3}$` matches 333 in 901-333

**9. Character Classes:** Match specific types of characters:

`\s`: whitespace

`\S`: non-whitespace

`\d`: digit

`\D`: non-digit

`\w`: word character (letters, digits, \_)

`\W`: non-word character

`\b`: word boundary

**Example:** `[abc]` matches a, b, or c

**10. Negated Character Class ([ ^ ]):** Matches characters not listed in the brackets.

**Example :** `[^abc]` -> matches any character except a, b, c

## ***Designing Finite Automata from Regular Expression***

*A regular expression is a simple and effective way to describe patterns in text, such as "any number of a's" written as  $a^*$  or "either a or b" written as  $a|b$ . A finite automaton is a basic machine that reads a string one character at a time and decides whether it matches a given pattern. Converting a regular expression into a finite automaton means transforming the pattern into a machine that can automatically check if a string follows that pattern. This process is very useful in computer science and practical applications.*

- It allows computers to automatically recognize and process text patterns.*
- It is used in tools like text editors and search features to find matching strings.*
- It plays an important role in programming languages and compilers, especially in lexical analysis.*

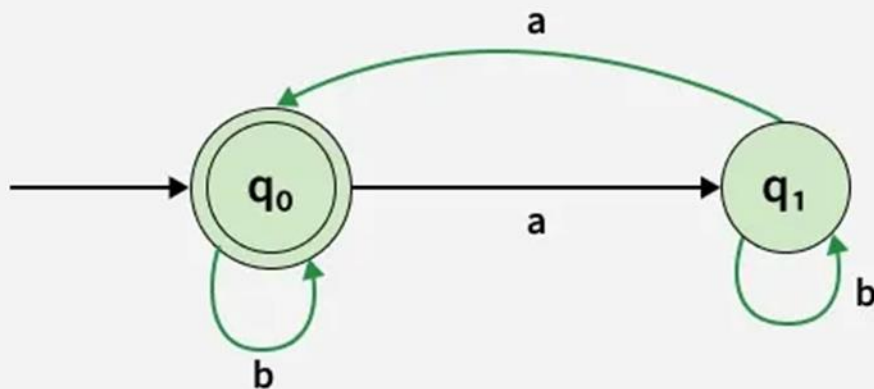
### ***Even number of a's***

*The regular expression for even number of a's is*

*$(b|ab^*ab^*)^*$*

*We can construct a finite automata as shown in figure below.*

**Figure 1**



The above automata will accept all strings which have even number of a's. For zero a's, it will be in  $q_0$  which is final state. For one 'a', it will go from  $q_0$  to  $q_1$  and the string will not be accepted. For two a's at any positions, it will go from  $q_0$  to  $q_1$  for 1st 'a' and  $q_1$  to  $q_0$  for second 'a'. So, it will accept all strings with even number of a's.

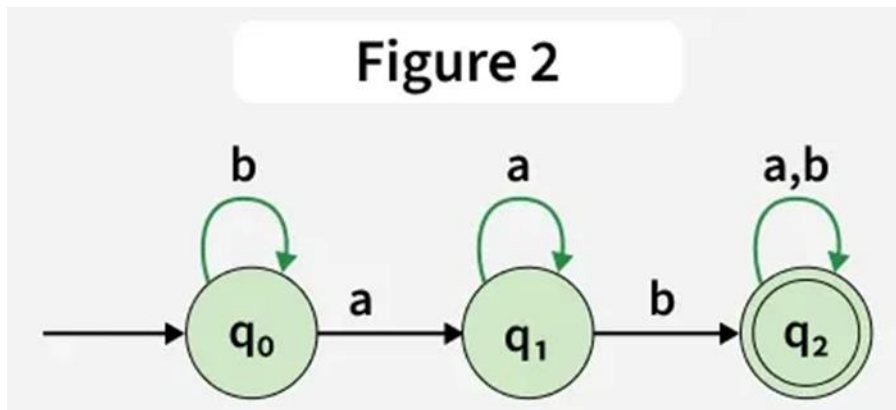
**String with 'ab' as substring**

The regular expression for strings with 'ab' as substring is

$(a|b)^*ab(a|b)^*$

We can construct finite automata as shown in figure below.

Figure 2



The above automata will accept all string which have 'ab' as substring. The automata will remain in initial state  $q_0$  for  $b$ 's. It will move to  $q_1$  after reading 'a' and remain in same state for all 'a' afterwards. Then it will move to  $q_2$  if 'b' is read. That means, the string has read 'ab' as substring if it reaches  $q_2$ .

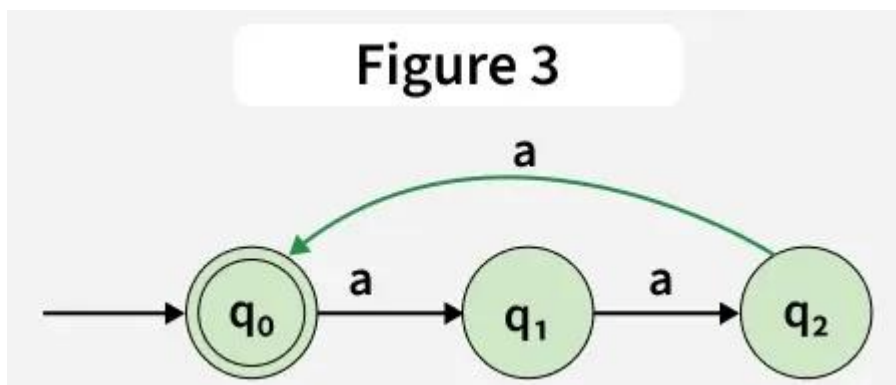
### String with count of 'a' divisible by 3

The regular expression for strings with count of a divisible by 3 is

$$\{a^{3n} \mid n \geq 0\}$$

We can construct automata as shown in Figure 3.

Figure 3



*The above automata will accept all string of form  $a^{3n}$ . The automata will remain in initial state  $q_0$  for ? and it will be accepted. For string 'aaa', it will move from  $q_0$  to  $q_1$  then  $q_1$  to  $q_2$  and then  $q_2$  to  $q_0$ . For every set of three a's, it will come to  $q_0$ , hence accepted. Otherwise, it will be in  $q_1$  or  $q_2$ , hence rejected.*

**Note :**

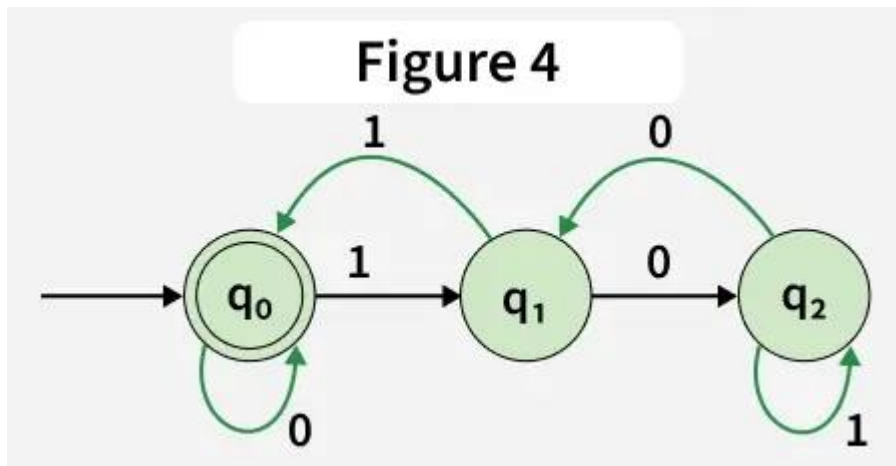
*If we want to design a finite automata with number of a's as  $3n+1$ , same automata can be used with final state as  $q_1$  instead of  $q_0$ . If we want to design a finite automata with language  $\{a^{kn} \mid n \geq 0\}$ ,  $k$  states are required. We have used  $k = 3$  in our example.*

**Binary numbers divisible by 3**

*The regular expression for binary numbers which are divisible by three is*

***$(0|1(01^*0)^*1)^*$***

*The examples of binary number divisible by 3 are 0, 011, 110, 1001, 1100, 1111, 10010 etc. The DFA corresponding to binary number divisible by 3 can be shown in Figure 4.*

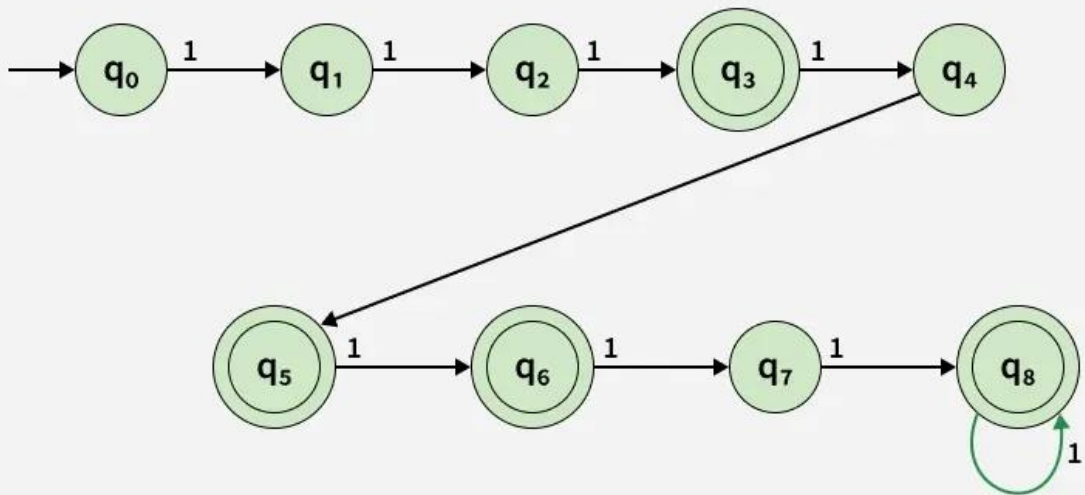


The above automata will accept all binary numbers divisible by 3. For 1001, the automata will go from  $q_0$  to  $q_1$ , then  $q_1$  to  $q_2$ , then  $q_2$  to  $q_1$  and finally  $q_2$  to  $q_0$ , hence accepted. For 0111, the automata will go from  $q_0$  to  $q_0$ , then  $q_0$  to  $q_1$ , then  $q_1$  to  $q_0$  and finally  $q_0$  to  $q_1$ , hence rejected.

**String with regular expression  $(111 + 11111)^*$**

The string accepted using this regular expression will have 3, 5, 6(111 twice), 8 (11111 once and 111 once), 9 (111 thrice), 10 (11111 twice) and all other counts of 1 afterwards. The DFA corresponding to given regular expression is given in Figure 5.

Figure 5



**What will be the minimum number of states for strings with odd number of a's**

**Solution :** The regular expression for odd number of a is

$b^*ab^*(ab^*ab^*)^*$

Corresponding automata is given in Figure 6 and minimum number of states are 2.

Figure 6

